

Aplicaciones Web II

Analista de Sistemas y Desarrollo de Software

Trabajo Práctico Integrador

Profesor

- Andres Senn

Equipo de desarrollo

- Bonavia Visconti Mateo – 43.133.957
- Ruffa Lucas Tomas – 46.033.791
- Galizzi Marcos – 45.307.659
- Lantieri Lucas – 41.681.985

ÍNDICE

Desarrollo de una plataforma web para la gestión de pedidos 3
1. **Introducción** 3
2. **Equipo de trabajo y roles** 3
3. **Descripción del proyecto** 4
Flujo funcional propuesto *4*
Alcance por etapas *4*
4. **Justificación y propuesta de valor** 4
5. **Objetivos** 5
Objetivo general *5*
Objetivos específicos *5*
6. **Planificación y cronograma** 5
Etapa 1 — Relevamiento y propuesta 6
Etapa 2 — Implementación del servidor Back-End 6
Etapa 3 — Administración del catálogo (CRUD) 6
Etapa 4 — Autenticación y seguridad 7
Etapa 5 — Despliegue a producción (opcional) 7
7. **Metodología de trabajo y herramientas** 7
Gestión de tareas *7*
Control de versiones *7*
8. **Arquitectura y stack tecnológico** 8
9. **Repositorio** 8
10. **Estado del desarrollo — Etapa 2** 8
Tecnologías a utilizar *8*
Endpoints propuestos *8*
Implementación del Front-End con el servidor *9*
Modularización del proyecto *9*
11. Estado del desarrollo — Etapa 3 ..... **10**
Ajustes sobre la Etapa 2 10
Estructura MVC del módulo de productos 11
Interfaz FrontEnd de administración 11
Endpoints propuestos 11
Base de datos PostgreSQL con Docker 12
Implementación de los endpoints del CRUD 12
Conexión de la interfaz con la API 13
Documentación de la API con Scalar 14
12. Estado del desarrollo — Etapa 4 ..... **14**
Ajustes sobre la Etapa 3 14
Configuración centralizada 14
Nueva organización modular del Back-End 15
Criterio uniforme de respuestas 15
Gestión de imágenes de productos 16
Endpoints actualizados 16

Desarrollo de una plataforma web para la gestión de pedidos

Propuesta de proyecto e informe de avance

1. Introducción

Este documento presenta la propuesta de trabajo y el estado de avance de un proyecto de desarrollo web pensado para un comercio que actualmente gestiona sus ventas mediante un catálogo estático y canales informales de contacto con sus clientes.

El equipo asume el rol de un equipo de desarrollo externo contratado para digitalizar y automatizar ese proceso: partiendo de un sitio institucional ya existente, el objetivo es construir un servidor propio y una capa de datos que conviertan el catálogo en una aplicación transaccional real, capaz de recibir y confirmar pedidos en línea.

El documento se irá actualizando a medida que avance el desarrollo, incorporando en cada etapa el detalle técnico correspondiente y manteniendo siempre la trazabilidad de las decisiones tomadas por el equipo.

2. Equipo de trabajo y roles

El equipo está compuesto por cuatro integrantes. Si bien cada uno tiene un rol principal para ordenar el trabajo, las tareas de desarrollo del servidor se reparten entre todos a lo largo del proyecto, de forma que cada avance quede reflejado en el trabajo conjunto del equipo y no en el de una sola persona.

Integrante	Rol principal	Responsabilidades
Bonavia Visconti Mateo	Coordinación general y Back-End	Coordinación del equipo y de los tiempos de entrega, modelado de la base de datos e implementación de la autenticación de usuarios.

Ruffa Lucas Tomas	Front-End	Adaptación de la interfaz existente a los nuevos datos dinámicos y validación de los flujos de navegación.
Galizzi Marcos	Back-End y Base de Datos	Configuración del servidor, integración con las fuentes de datos, gestión del repositorio y control de versiones.
Lantieri Lucas	Calidad y Documentación	Pruebas de las funcionalidades desarrolladas, documentación técnica y colaboración en el desarrollo del servidor.

3. Descripción del proyecto

El proyecto consiste en evolucionar el sitio web de toma de pedidos de camisetas de fútbol retro y actuales —hasta ahora un catálogo estático— hacia una aplicación web con servidor propio, capaz de servir la información de productos de forma dinámica y de procesar pedidos de punta a punta.

La interfaz conserva la estructura ya validada con el comercio: una presentación institucional, un catálogo organizado por categorías que ahora se alimentará desde una fuente de datos externa, y un formulario final que actúa como checkout para confirmar la orden.

Flujo funcional propuesto

- El cliente ingresa al sitio y accede al catálogo de productos.
- Selecciona una categoría y visualiza los productos disponibles, obtenidos dinámicamente desde el servidor.
- Agrega productos a su pedido y completa el formulario de checkout.
- El pedido queda registrado y disponible para que el comercio lo gestione, reemplazando el intercambio manual por WhatsApp.

Alcance por etapas

El desarrollo se aborda de forma incremental: primero se construye el servidor que sirve el sitio y expone los datos desde una fuente externa; luego se incorporan las operaciones de creación, lectura, actualización y eliminación necesarias para administrar el catálogo; y finalmente se agrega un sistema de autenticación y control de acceso para proteger esas operaciones administrativas.

4. Justificación y propuesta de valor

El catálogo actual presenta la información de forma estática: cualquier cambio de precio, descripción o disponibilidad de stock requiere modificar directamente el código fuente del sitio, lo cual es lento y propenso a errores.

A su vez, la recepción de pedidos depende de canales informales como WhatsApp. Este tipo de flujo manual suele generar demoras en la respuesta, errores de carga por parte de quien toma el pedido y, en horarios de alta demanda, pérdida directa de ventas por falta de capacidad de atención simultánea.

La propuesta de valor del proyecto es centralizar el catálogo en una base de datos administrable y automatizar la recepción de pedidos a través de un flujo web propio, reduciendo la dependencia de la gestión manual y sentando las bases para futuras mejoras (administración propia del catálogo, autenticación de usuarios, panel de gestión de pedidos).

5. Objetivos

Objetivo general

Transformar el sitio de toma de pedidos, actualmente estático, en una aplicación web transaccional con servidor propio, capaz de gestionar dinámicamente el catálogo y de procesar pedidos en línea.

Objetivos específicos

- Conservar la interfaz Front-End ya desarrollada, adaptándola para consumir datos dinámicos.
- Implementar un servidor propio que sirva el sitio y centralice el acceso a los datos del catálogo.
- Conectar el catálogo a una fuente de datos externa, evitando la dependencia de archivos estáticos.
- Incorporar progresivamente operaciones de administración (CRUD) sobre el catálogo.
- Implementar autenticación y control de acceso para proteger las operaciones administrativas.
- Reducir la dependencia de canales informales para la recepción de pedidos.
- Mantener una organización de trabajo ordenada, con control de versiones y seguimiento de tareas por integrante.

6. Planificación y cronograma

TP	A desarrollar	Fecha límite
TP1	Propuesta por escrito	**28/08**
TP2	Servidor para Front-End	**18/09**
TP3	CRUD	**16/10**

TP4	Login y seguridad del CRUD	**06/11**

El proyecto se organiza en cinco etapas incrementales. Cada una agrupa un conjunto de tareas concretas con un responsable asignado, sin perjuicio de que el equipo colabore de forma cruzada cuando es necesario.

Etapa 1 — Relevamiento y propuesta

Estado: Hecho · entrega 28/08

Tarea	Responsable
Definición del equipo y los roles de trabajo	Todo el equipo
Relevamiento del alcance funcional y la justificación del proyecto	Bonavia Visconti Mateo
Redacción de la propuesta de desarrollo	Lantieri Lucas
Selección de las herramientas de gestión y control de versiones	Galizzi Marcos

Etapa 2 — Implementación del servidor Back-End

Estado: Hecho · entrega 18/09

Tarea	Responsable
Configuración del servidor con Node.js y Express	Galizzi Marcos
Conexión del servidor a la fuente de datos externa (MockAPI)	Galizzi Marcos
Organización modular de archivos y carpetas del proyecto	Lantieri Lucas
Definición y prueba de los endpoints iniciales	Bonavia Visconti Mateo
Integración de los nuevos endpoints con el Front-End existente	Ruffa Lucas Tomas
Redacción del informe de avance de esta etapa	Lantieri Lucas

Etapa 3 — Administración del catálogo (CRUD)

Estado: Hecho · entrega 23/10

Tarea	Responsable
Organización del proyecto con un patrón MVC modular	Galizzi Marcos
Implementación de Router y de los 5 endpoints del CRUD	Galizzi Marcos / Lantieri Lucas
Migración de los datos a una base de datos real (PostgreSQL o MongoDB)	Bonavia Visconti Mateo
Desarrollo de la interfaz Front-End de administración	Ruffa Lucas Tomas
Documentación de la API con Scalar	Lantieri Lucas

Etapa 4 — Autenticación y seguridad

Estado: Por hacer · entrega 06/11

Tarea	Responsable
Persistencia de usuarios registrados en base de datos	Bonavia Visconti Mateo
Hashing de contraseñas	Bonavia Visconti Mateo
Autenticación mediante JWT y cookies	Bonavia Visconti Mateo / Galizzi Marcos
Habilitación justificada de CORS	Galizzi Marcos
Uso de variables de entorno para datos sensibles	Galizzi Marcos
Ampliación de la documentación de la API	Lantieri Lucas

Etapa 5 — Despliegue a producción (opcional)

Estado: a definir según el avance del proyecto

7. Metodología de trabajo y herramientas

Gestión de tareas

El equipo organiza y da seguimiento a las tareas mediante GitHub Projects. El tablero se divide en cinco columnas —Pendiente, En progreso, En revisión, Bloqueada y Finalizada— y cada tarea se asigna a un responsable y se etiqueta según el tipo de trabajo (Back-End, Front-End, documentación o testing), lo que permite visualizar de forma clara el aporte de cada integrante.

Control de versiones

Se utiliza Git como sistema de control de versiones y GitHub como plataforma para alojar el repositorio. Cada nueva funcionalidad se desarrolla en una rama propia; al finalizarla, se abre un pull request que es revisado por al menos otro integrante del equipo antes de fusionarlo a la rama principal.

Esta dinámica busca mantener un historial de cambios ordenado y trazable, además de que las tareas del tablero queden vinculadas a los commits y pull requests correspondientes.

8. Arquitectura y stack tecnológico

El Front-End no se conecta directamente a la fuente de datos externa. Se implementa un servidor propio que actúa como intermediario entre la interfaz y los datos:

Front-End (HTML / CSS / JS) → Servidor propio (Node.js + Express) → Fuente de datos externa (MockAPI)

- El servidor sirve los archivos estáticos del Front-End desarrollado previamente.
- Expone rutas propias que consultan los datos externos y los devuelven al Front-End ya procesados.
- En etapas posteriores, la fuente de datos externa se reemplaza por una base de datos propia (PostgreSQL o MongoDB), sin que el Front-End deba modificarse.

9. Repositorio

El repositorio del proyecto contiene, y se actualiza de forma incremental en cada etapa:

- Código fuente del proyecto.
- Este documento, con la propuesta original y los informes de avance de cada etapa.
- Descripción de los endpoints desarrollados, con su método y una breve referencia.
- A partir de la etapa de administración del catálogo, el archivo SQL o de MongoDB es necesario para replicar la base de datos.

10. Estado del desarrollo — Etapa 2

Esta etapa se encuentra en curso. A continuación se detalla la propuesta técnica sobre la que se está trabajando; esta sección se irá completando con el detalle final a medida que avance la implementación.

Tecnologías a utilizar

- Node.js como entorno de ejecución del servidor.
- Express.js como framework para definir el servidor y sus rutas.
- MockAPI como fuente de datos externa para categorías y productos.

Endpoints propuestos

En línea con el catálogo definido en la propuesta original (liga y productos), se plantean los siguientes dos endpoints de lectura:

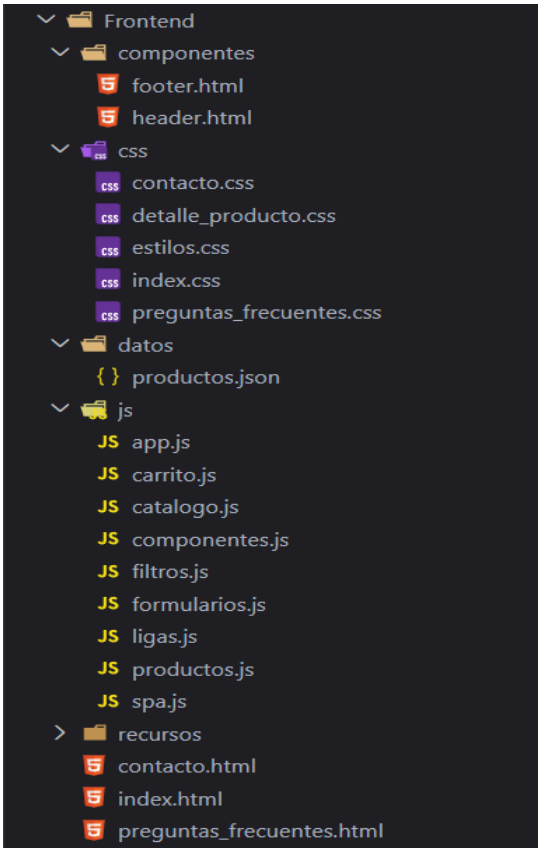
Método	Ruta	Descripción
GET	(base-url)/liga	Obtiene todas las ligas disponibles para filtrar los productos.
GET	(base-url)/productos	Obtiene todos los productos disponibles.

Implementación del Front-End con el servidor

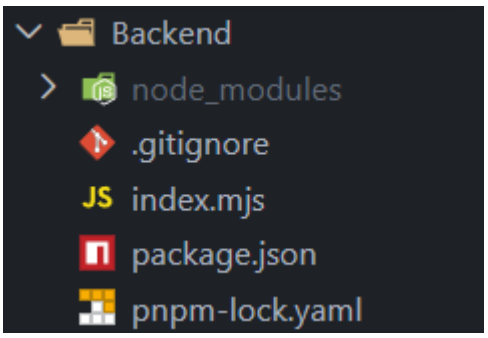
Se modificaron los archivos '**productos.js**' y '**liga.js**' para que, en lugar de leer productos.json de forma local, consuman los recursos que les provee el servidor. Cuando se carga la página web, se solicitan las ligas y los productos a los respectivos endpoints para mostrarlos por pantalla y que el usuario pueda filtrar según lo seleccionado.

Modularización del proyecto

El proyecto se compone de dos carpetas principales: '**Frontend**' y '**Backend**'



Frontend se constituye de las carpetas componentes (elementos reutilizados en todas las secciones de la página), css (estilos), datos (antigua fuente de obtención de datos de los productos), js (funcionalidad general de la web), recursos (elementos gráficos) y tres archivos .html que componen las secciones iniciales de la app.



Backend incluye los archivos necesarios para iniciar el servidor y gestionar las peticiones a los endpoints.

11. Estado del desarrollo — Etapa 3

A continuación se detallan los ajustes realizados sobre la etapa anterior, la migración del catálogo a una base de datos PostgreSQL y la implementación de los endpoints de lectura, alta, baja y modificación, junto con su conexión a la interfaz de administración.

Ajustes sobre la Etapa 2

A partir de las observaciones recibidas en la entrega anterior, se realizaron las siguientes modificaciones:

- Se configuró Express para servir los archivos estáticos de la carpeta '**Frontend**', de modo que el sitio completo se accede desde el mismo servidor que expone los endpoints.
- En '**productos.js**' y '**ligas.js**' se reemplazaron las direcciones absolutas (http://localhost:3000/...) por rutas relativas (/productos y /liga). Al compartir el mismo origen, la interfaz ya no depende del host ni del puerto en el que se ejecute la aplicación.
- En '**index.mjs**' se agruparon al inicio las direcciones de la fuente de datos externa (MockAPI), centralizando su configuración.
- Se incorporó un middleware de manejo de errores que registra en consola los errores no controlados y responde al cliente con código 500 y un mensaje en formato JSON.

Estructura MVC del módulo de productos

Para organizar el proyecto bajo el patrón MVC, se creó dentro de '**Backend**' la carpeta '**modulos/productos**' con los siguientes archivos:

Archivo A	Responsabilidad
productos.modelo.mjs	Acceso a los datos de los productos.
productos.controlador.mjs	Lógica que procesa cada petición.
productos.vista.mjs	Formato de las respuestas enviadas al cliente.
productos.rutas.mjs	Definición de las rutas del módulo mediante Router.

La lógica que anteriormente residía en '**index.mjs**' se trasladó a estos archivos: el servidor se limita a configurar Express y a montar el router del módulo en '/api/v1/productos'.

Interfaz FrontEnd de administración

Se desarrolló la interfaz desde la cual se administrará el catálogo, integrada en la página principal:

- Un botón "Crear producto", ubicado sobre el listado, que abre un formulario para dar de alta un nuevo producto.
- En cada tarjeta del catálogo, dos botones para editar y eliminar el producto correspondiente.
- Tres ventanas de diálogo: confirmación de eliminación, edición de producto (con los datos actuales precargados) y creación de producto. Los formularios incluyen nombre, precio, descripción, etiqueta, año e imágenes.

Esta funcionalidad se reúne en el nuevo archivo '**admin.js**', que se inicializa desde '**app.js**' junto con el resto del sitio; los estilos se agregaron en '**index.css**' y '**estilos.css**'.
Las acciones de crear y eliminar ya se encuentran conectadas con la API (ver 'Conexión de la interfaz con la API'); la edición es por ahora solo visual: al confirmar, el diálogo se cierra sin enviar datos.

Endpoints propuestos

Para completar la administración del catálogo, se plantean los siguientes endpoints sobre el recurso productos:

Método	Ruta	Descripción
GET	(baseurl)/api/v1/productos	Obtiene todos los productos disponibles.
GET	(baseurl)/api/v1/productos/{id}	Obtiene un producto según su id.

POST	(baseurl)/api/v1/productos	Crea un nuevo producto.
PUT	(baseurl)/api/v1/productos/{id}	Modifica los datos de un producto existente.
DELETE	(baseurl)/api/v1/productos/{id}0	Elimina un producto del catálogo.

Base de datos PostgreSQL con Docker

El catálogo dejó de depender de MockAPI y pasó a almacenarse en una base de datos PostgreSQL que se ejecuta en un contenedor Docker. Dentro de '**Backend/postgre-sql**' se incluyen los siguientes archivos:

- '**tienda.sql**': script que crea las tablas ligas y productos y las carga con los datos iniciales del catálogo.
- '**Dockerfile**': imagen basada en postgres que copia el script en la carpeta de inicialización, de modo que la base de datos se crea automáticamente al iniciar el contenedor.
- '**comandos.txt**': comandos para construir la imagen y levantar el contenedor.

La base de datos se organiza en dos tablas relacionadas:

Tabla	Campos
ligas	id (clave primaria) y nombre (único).
productos	id (clave primaria), nombre, descripcion, precio (mayor o igual a 0), etiqueta, anio, liga_id (referencia a ligas), imagenes, talles y tags.

La conexión se centraliza en '**conexion.bd.mjs**', que crea un pool de conexiones con la librería pg a partir de variables de entorno (BD_HOST, BD_USER, BD_PASS, BD_BD y BD_PORT). El archivo '**.env.example**' documenta las variables necesarias sin exponer datos sensibles, y el archivo .env real se excluye del repositorio mediante '**.gitignore**'.

Implementación de los endpoints del CRUD

Los endpoints se implementaron en el módulo '**modulos/productos**', respetando el patrón MVC descrito anteriormente:

- '**productos.modelo.mjs**': consultas SQL parametrizadas contra PostgreSQL para listar, buscar por id, insertar y borrar productos.
- '**productos.controlador.mjs**': valida los datos recibidos (id numérico, nombre obligatorio, precio válido) y decide la respuesta a enviar.
- '**productos.vista.mjs**': define el formato de las respuestas JSON y los códigos de estado.
- '**productos.rutas.mjs**': asocia cada método y ruta con su controlador mediante Router; '**index.mjs**' lo monta en '/api/v1/productos'.

Las rutas se agruparon bajo el prefijo '/api/v1' y las rutas anteriores que consultaban MockAPI quedaron comentadas en '**index.mjs**'. Los errores se informan con un cuerpo JSON de la forma { error: mensaje } y los no controlados continúan siendo atendidos por el

middleware de errores (código 500). Se habilitó CORS mediante el middleware cors.
Estado de cada endpoint:

Método	Ruta	Comportamiento
GET	(base-url)/api/v1/productos	Devuelve todos los productos ordenados por id (200).
GET	(base-url)/api/v1/productos/:{id}	Devuelve un producto según su id (200). Responde 400 si el id no es un entero positivo y 404 si no existe.
POST	(base-url)/api/v1/productos	Crea un producto a partir de un formulario multipart con nombre, descripción, precio, etiqueta, año, liga e imágenes (hasta 5, almacenadas con multer en Frontend/recursos/subidas). Exige nombre y precio mayor o igual a 0 (400 en caso contrario) y responde 201 con el producto creado.
DELETE	(base-url)/api/v1/productos/{id}	Elimina un producto (204 sin cuerpo). Responde 400 si el id es inválido y 404 si no existe.
GET	(base-url)/api/v1/ligas	Devuelve todas las ligas desde la base de datos (200).
PUT	(base-url)/api/v1/productos/{id}	Modifica un producto a partir de un formulario multipart con nombre, descripción, precio, etiqueta, año, liga e imágenes (hasta 5, almacenadas con multer en Frontend/recursos/subidas). Exige nombre y precio mayor o igual a 0 (400 en caso contrario) y responde 200 con el producto creado.

Se agregó el archivo '**test.http**' en '**Backend**' con peticiones de prueba para los endpoints implementados (lecturas, alta con datos de ejemplo y baja).

Conexión de la interfaz con la API

Los archivos del Front-End pasaron a consumir los nuevos endpoints:

- '**productos.js**' y '**ligas.js**' solicitan los datos a '/api/v1/productos', '/api/v1/productos/:id' y '/api/v1/ligas', con la dirección base definida en una constante.
- **Alta de producto:** el formulario de creación envía por POST los datos y las imágenes (FormData) a la API. Si la respuesta es exitosa, se cierra el diálogo y se recarga el catálogo para mostrar el nuevo producto; si no, se informa al usuario el mensaje de error devuelto por el servidor.
- **Baja de producto:** al confirmar en el diálogo de eliminación, se envía un DELETE con el id del producto seleccionado. Ante una respuesta exitosa (204) se cierra el diálogo y se recarga el catálogo sin el producto borrado.
- **Modificación de producto:** el formulario de edición envía por PUT los datos y las

imágenes (FormData) a la API. Si la respuesta es exitosa, se cierra el diálogo y se recarga el catálogo para mostrar el producto modificado; si no, se informa al usuario el mensaje de error devuelto por el servidor.

- Si el servidor no responde, la interfaz muestra un aviso en lugar de fallar silenciosamente.

Documentación de la API con Scalar

https://aw-ii.apidocumentation.com/api-reference

12. Estado del desarrollo — Etapa 4

Esta etapa se encuentra en curso. A continuación se detallan las correcciones realizadas a partir de las observaciones recibidas en la entrega de la Etapa 3, que reorganizan el servidor y unifican su comportamiento antes de completar la autenticación y la seguridad del CRUD.

Ajustes sobre la Etapa 3

A partir de las observaciones recibidas en la entrega anterior, se realizaron las siguientes modificaciones:

- **Lógica fuera de 'index.mjs':** las rutas de ligas y de autenticación (inicio de sesión, registro y JWT) se trasladaron a módulos propios con el mismo patrón MVC que el módulo de productos.
- **Trabajo en ramas:** las correcciones se desarrollaron en la rama 'correcciones-tp3' y se integran a la rama principal mediante un pull request revisado por el equipo.
- **Configuración centralizada:** las rutas, direcciones y variables de entorno dejaron de estar escritas en cada archivo y se definen en un único archivo de configuración.
- **Respuestas uniformes:** todos los endpoints responden en formato JSON con un mismo criterio, incluida la eliminación de productos, que antes respondía 204 sin cuerpo.
- **Imágenes:** las imágenes de los productos se guardan en una única carpeta y se borran del disco cuando dejan de utilizarse.

Configuración centralizada

Se creó el archivo '**config.mjs**' en '**Backend**', que lee las variables de entorno y expone los valores que utiliza el resto del servidor:

Constante	Contenido
PUERTO	Puerto en el que escucha el servidor (variable PORT).
JWT_SECRET	Clave con la que se firman y verifican los tokens de sesión.
BD	Datos de conexión a PostgreSQL, utilizados por 'conexion.bd.mjs'.

CARPETA_FRONTEND	Ruta absoluta de la carpeta que sirve Express como archivos estáticos.
CARPETA_IMAGENES	Carpeta del disco donde se guardan las imágenes de los productos.
URL_IMAGENES	Prefijo con el que el Front-End accede a esas imágenes.

Las rutas se calculan a partir de la ubicación del propio archivo, por lo que el servidor funciona correctamente sin importar desde qué carpeta se ejecute. Del lado del cliente se agregó '**config.js**', que define la dirección de la API utilizada por '**productos.js**', '**ligas.js**' y '**admin.js**'.

Nueva organización modular del Back-End

'**index.mjs**' se limita a configurar Express, montar los routers de cada módulo y registrar el middleware de errores. Cada módulo se encuentra dentro de '**modulos**' y se compone de los archivos modelo, vista, controlador y rutas:

Módulo	Ruta base	Responsabilidad
productos	/api/v1/productos	CRUD del catálogo y gestión de las imágenes de cada producto ('productos.imagenes.mjs').
ligas	/api/v1/ligas	Consulta de las ligas utilizadas para filtrar el catálogo.
auth	/api/v1	Registro de usuarios, inicio de sesión con JWT en cookie y verificación de la sesión activa.

Además, el archivo compartido '**respuestas.mjs**' define las funciones que utilizan las vistas de todos los módulos para responder al cliente.

Criterio uniforme de respuestas

Todas las respuestas de la API siguen el mismo formato JSON según su tipo:

Tipo	Formato	Uso
Datos	El recurso solicitado	Listado de productos y ligas, consulta, alta y modificación de un producto.
Acción	{ mensaje, ... }	Eliminación de un producto, inicio de sesión, registro y consulta de la sesión.
Error	{ error }	Cualquier respuesta con código 4xx o 5xx.

La eliminación de un producto pasó a responder 200 con el mensaje de confirmación y los datos del producto eliminado, de modo que el Front-End recibe información sobre la operación realizada y la muestra al usuario.

Gestión de imágenes de productos

El manejo de imágenes se separó del controlador en '**productos.imagenes.mjs**':

- **Carpeta única:** las imágenes subidas se guardan en 'Frontend/recursos/Camisetas', junto a las imágenes iniciales del catálogo, con un nombre único que conserva su extensión.
- **Borrado al eliminar:** al eliminar un producto se borran del disco todas sus imágenes.
- **Borrado al reemplazar:** si al modificar un producto se suben imágenes nuevas, se borran las anteriores.
- **Limpieza ante errores:** si el producto no llega a guardarse (datos inválidos o error de la base de datos), se borran las imágenes recién subidas.
- **Seguridad:** solo se eliminan archivos ubicados dentro de la carpeta de imágenes.

Endpoints actualizados

Estado de los endpoints que cambiaron o se incorporaron en esta etapa:

Método	Ruta	Comportamiento
DELETE	(base-url)/api/v1/productos/{id}	Elimina un producto y sus imágenes. Responde 200 con { mensaje, producto }, 400 si el id es inválido y 404 si no existe.
PUT	(base-url)/api/v1/productos/{id}	Modifica un producto. Si se envían imágenes nuevas, reemplaza y borra las anteriores. Responde 200 con el producto modificado.
POST	(base-url)/api/v1/login	Inicia sesión y guarda el token en una cookie (200). Responde 400 si faltan datos y 401 si el correo o la contraseña son incorrectos.
GET	(base-url)/api/v1/sesion	Devuelve el usuario de la sesión activa (200) o 401 si no hay sesión.
POST	(base-url)/api/v1/registrarse	Crea un usuario con la contraseña hasheada (201). Responde 400 si faltan datos y 409 si el correo ya está registrado.